IT STUDY HUB

COMPLETE C++ PROGRAMMING SERIES

MODULE 2: VARIABLES, DATA TYPES & OPERATORS

Authors: Mithila Rani Saha & Dithsa Dutta

Website: www.itstudyhub.dpdns.org

Module Overview

1. About This Module

Welcome to **Module 2: Variables, Data Types & Operators** in the IT Study Hub Complete C++ Programming Series. Building directly on the foundational concepts introduced in Module 1, this module transitions you from writing basic static print statements to managing memory storage, handling diverse data categories, and performing complex mathematical and

logical evaluations. Authored by Mithila Rani Saha and Dithsa Dutta, this handbook provides a deep dive into how C++ interacts with physical computer memory at a granular level.

2. Learning Outcomes

By completing this module, you will be able to:

- Declare, initialize, and manage variables and constants effectively within specified scopes.
- Distinguish between primitive data types (`int`, `float`, `double`, `char`, `bool`, `void`, `wchar_t`) and modifiers (`short`, `long`, `signed`, `unsigned`).
- Utilize the `sizeof` operator to inspect memory footprints across different architectures.
- Perform both implicit and explicit type casting safely without losing critical precision.
- Master arithmetic, assignment, relational, logical, bitwise, increment/decrement, and ternary operators.
- Comprehend operator precedence rules and evaluate complex mathematical expressions deterministically.

3. Module Coverage

This module comprehensively covers:

- **Memory Foundations:** Variables, Constants, Scope rules, and Storage visualization.
- **Data Typology:** Fundamental primitives, type modifiers, and memory sizing (`sizeof`).
- **Data Transformation:** Implicit promotions and explicit casting mechanisms.
- **Operator Matrix:** Complete operator taxonomy including bitwise operations and precedence tables.
- **Hands-On Project:** Building a fully robust, interactive command-line C++ calculator.
- **Rigorous Assessments:** 15+ practice questions, 20+ viva questions with answers, 10+ technical interviews, and coding challenges.

4. Main Notes: Theoretical Foundations

To master C++, you must understand how data is represented in memory and how operations manipulate those bits. Below is the detailed breakdown of variables, data types, and operators.

Concept: Variables and Constants

Simple Explanation: A variable is a named storage location in computer memory (RAM) used to hold data that can change during program execution. A constant, declared using the `const` keyword or `#define`, represents fixed data that cannot be modified once set.

> ***Real-Life Analogy:*** *Think of a variable as a labeled storage box in a warehouse where you can throw away old items and put in new ones. A constant is like a permanent engraved plaque fixed to a wall—the content is locked forever the moment it is manufactured.*

> **Syntax:**
> **`datatype variable_name = initial_value;`**
> **`const datatype CONSTANT_NAME = value;`**

```
#include
using namespace std;

int main() {
    int score = 45; // Variable
    score = 50;     // Can be changed

    const double PI = 3.14159; // Constant
    // PI = 3.0; // Error: assignment of read-only variable

    cout << "Score: " << score << ", PI: " << PI << endl;
    return 0;
}
```

```
Output:
Score: 50, PI: 3.14159
```

Line-by-Line Breakdown:

- **int score = 45;**: Allocates memory for an integer named `score` and initializes it.
- **score = 50;**: Overwrites the existing value in memory.
- **const double PI = 3.14159;**: Declares a read-only constant double precision floating-point number.
- **cout ...**: Outputs both values.

Real-World Application: In banking applications, a user's account balance is a variable (changes with every transaction), whereas the bank's routing number or transaction fee percentage is stored as a constant.

Common Mistakes: Trying to modify a `const` variable after its initial declaration, which triggers a compile-time error.

Best Practices: Always use UPPERCASE naming conventions for constants (e.g., `MAX_RETRIES`, `PI`) to distinguish them clearly from regular lowercase or camelCase variables.

Actionable Practice: Declare a constant for gravitational acceleration (`9.81`) and a variable for time elapsed, then calculate falling distance.

Concept: Primitive Data Types (int, float, double, char, bool, void, wchar_t)

Simple Explanation: C++ provides several built-in primitive data types. `int` handles whole numbers, `float` and `double` handle decimals (with `double` offering twice the precision), `char`

stores single characters, `bool` holds truth values (`true` or `false`), `void` represents the absence of type, and `wchar_t` stores wide characters for international character sets.

> ***Real-Life Analogy:*** *Think of data types as different containers in a kitchen: a small cup for single drops (`char`), a bucket for whole liters (`int`), and specialized measuring flasks for precise chemical liquids (`double`). Putting liquid into a solid container incorrectly spills or corrupts data.*

Syntax:

```
int a;
float b;
double c;
char d;
bool e;
wchar_t f;
```

```
#include
using namespace std;

int main() {
    int age = 25;
    float temperature = 98.6f;
    double piPrecise = 3.14159265359;
    char grade = 'A';
    bool isPassed = true;

    cout << "Age: " << age << endl;
    cout << "Temp: " << temperature << endl;
    cout << "Pi: " << piPrecise << endl;
    cout << "Grade: " << grade << endl;
    cout << "Passed: " << isPassed << endl;
    return 0;
}
```

Output:

```
Age: 25
Temp: 98.6
Pi: 3.14159
```

```
Grade: A
Passed: 1
```

Line-by-Line Breakdown:

- **float temperature = 98.6f;**: The `f` suffix tells the compiler to treat the literal as a float instead of defaulting to a double.
- **bool isPassed = true;**: Prints as `1` because booleans output as numeric integers by default in standard streams.
- Data types dictate exactly how many bytes of RAM are allocated for each variable.

Real-World Application: Scientific calculations use `double` because tiny rounding errors in `float` can cause rockets to miss their orbits.

Common Mistakes: Assigning floating-point numbers directly to `int` variables without realizing that the fractional part is silently truncated.

Best Practices: Default to `double` for decimal numbers unless memory is severely constrained (like in embedded microcontrollers).

Actionable Practice: Write a program declaring one variable of each primitive type and print their sizes using `sizeof`.

Concept: Data Type Modifiers (short, long, signed, unsigned)

Simple Explanation: Modifiers alter the meaning of base data types to fit specific memory and range requirements. `short` and `long` adjust storage size, while `signed` (the default) allows positive and negative numbers, and `unsigned` restricts values strictly to zero and positive numbers, doubling the upper positive range.

> ***Real-Life Analogy:*** *Imagine a digital odometer in a car. A signed odometer can show forward and reverse mileage (positive and negative), whereas an unsigned odometer only counts forward distance, allowing it to record twice as high a maximum mileage before rolling over.*

```
Syntax:
unsigned int positiveOnly = 4000000000U;
long long bigNumber = 9000000000000LL;
short smallInt = 32000;
```

```
#include
using namespace std;

int main() {
    unsigned int uVar = 4000000000U;
    signed int sVar = -15000;
    long lVar = 2147483647L;

    cout << "Unsigned: " << uVar << endl;
    cout << "Signed: " << sVar << endl;
    cout << "Long: " << lVar << endl;
    return 0;
}
```

```
Output:
Unsigned: 4000000000
Signed: -15000
Long: 2147483647
```

Line-by-Line Breakdown:

- **unsigned int**: Cannot store negative numbers, but handles large positive values up to ~4 billion.
- **long lVar**: Guarantees at least 32 bits (and often 64 bits on modern 64-bit systems) of storage.

Real-World Application: Counting items in an inventory warehouse can never be negative, so using an `unsigned int` prevents impossible negative counts and doubles your maximum capacity.

Common Mistakes: Storing a negative number in an `unsigned` variable, causing an integer overflow wrap-around into massive positive numbers.

Best Practices: Use `unsigned` types exclusively when negative values are logically impossible (e.g., sizes, indices, counts).

Actionable Practice: Test what happens when you assign `-1` to an `unsigned int` and print the result.

Concept: The sizeof Operator

Simple Explanation: The `sizeof` operator is a compile-time unary operator that determines the size in bytes allocated for a specific variable or data type. Because data type sizes can vary across hardware architectures (32-bit vs 64-bit), `sizeof` ensures platform independence.

> ***Real-Life Analogy:*** *Think of `sizeof` as stepping on a weighing scale designed specifically for data containers. No matter what number is inside the box, the scale tells you exactly how many physical bytes of shelf space that container occupies.*

Syntax:

```
sizeof(datatype);
sizeof(variable);
```

```
#include
using namespace std;

int main() {
    cout << "Size of char: " << sizeof(char) << " byte(s)" << endl;
    cout << "Size of int: " << sizeof(int) << " bytes" << endl;
    cout << "Size of float: " << sizeof(float) << " bytes" << endl;
```

```
    cout << "Size of double: " << sizeof(double) << " bytes" <<
endl;
    return 0;
}
```

Output:
```
Size of char: 1 byte(s)
Size of int: 4 bytes
Size of float: 4 bytes
Size of double: 8 bytes
```

Line-by-Line Breakdown:

- **sizeof(int)**: Returns 4 on most modern systems, meaning integers take up 32 bits of RAM.
- **sizeof(double)**: Returns 8, indicating 64 bits of precision storage.

Real-World Application: When writing low-level network drivers or file parsers, developers must know exact byte sizes to pack data structures correctly across network sockets.

Common Mistakes: Assuming data type sizes are identical across all operating systems and hardware architectures (e.g., embedded chips vs mainframes).

Best Practices: Use `sizeof` when allocating dynamic memory buffers to ensure portability across different compiler targets.

Actionable Practice: Write a snippet finding the size of a `bool` and a `long long`.

Concept: Type Casting (Implicit and Explicit Conversion)

Simple Explanation: Type casting is the process of converting a variable from one data type to another. Implicit conversion is performed automatically by the compiler (type promotion), while explicit conversion (type casting) is forced manually by the programmer.

> ***Real-Life Analogy:*** *Implicit conversion is like pouring water from a small measuring cup into a large bucket—it fits naturally without spilling. Explicit casting is like trying to force a large bucket of water into a coffee mug: you have to intentionally squeeze and cut down the volume, risking data loss.*

```
Syntax:
// Implicit
int a = 5;
double b = a;

// Explicit (C-style & Functional)
double pi = 3.14;
int intPi = (int)pi; // C-style
int intPi2 = static_cast(pi); // Modern C++
```

```
#include
using namespace std;

int main() {
    // Implicit promotion
    int numInt = 10;
    double numDouble = numInt;

    // Explicit casting
    double pi = 3.14159;
    int truncatedPi = static_cast(pi);

    cout << "Implicit double: " << numDouble << endl;
    cout << "Explicit int: " << truncatedPi << endl;
```

```
    return 0;
}
```

```
Output:
Implicit double: 10
Explicit int: 3
```

Line-by-Line Breakdown:

- **double numDouble = numInt;**: Automatically promotes the integer `10` to floating-point `10.0`.
- **static_cast<int>(pi)**: Explicitly instructs the compiler to strip away the decimal fraction, converting `3.14159` to `3`.

Real-World Application: Financial systems implicitly promote integers to floating-point values when calculating interest rates, but explicitly cast floating-point totals back to integers when printing exact cents.

Common Mistakes: Forgetting that explicit downcasting truncates fractional values, turning `9.99` into `9` instead of rounding.

Best Practices: Prefer modern C++ cast operators like `static_cast()` over legacy C-style casts `(type)value` for better safety and readability.

Actionable Practice: Divide two integers (`7 / 2`) and use explicit casting to ensure the output displays `3.5` instead of integer division `3`.

Concept: Arithmetic and Assignment Operators

Simple Explanation: Arithmetic operators (`+`, `-`, `*`, `/`, `%`) perform basic mathematical computations. Assignment operators (`=`, `+=`, `-=`, `*=`, `/=`) assign values to variables and provide shorthand updates.

> ***Real-Life Analogy:*** *Arithmetic operators are basic math tools on a calculator. Assignment operators are like ledger adjustments: instead of writing 'Take current balance, add 50, and store it back', you simply write '+=' to update the ledger in one stroke.*

Syntax:

```
int sum = a + b;
int remainder = a % b;
x += 5; // equivalent to x = x + 5;
```

```
#include
using namespace std;

int main() {
    int a = 10, b = 3;

    cout << "Addition: " << a + b << endl;
    cout << "Modulus (Remainder): " << a % b << endl;

    a += 5; // a is now 15
    cout << "Compound Assignment (a += 5): " << a << endl;
    return 0;
}
```

Output:

```
Addition: 13
Modulus (Remainder): 1
Compound Assignment (a += 5): 15
```

Line-by-Line Breakdown:

- **a % b**: Computes the remainder of integer division (`10 / 3 = 3` with remainder `1`).
- **a += 5**: Adds 5 to `a` and updates its storage value instantly.

Real-World Application: Modulus operators (`%`) are used in computer algorithms to check if a number is even or odd (`num % 2 == 0`), or to wrap elements around circular buffers.

> **Common Mistakes:** Applying the modulus operator `%` to floating-point numbers (`double` or `float`), which causes compilation errors because modulus only applies to integers.

> **Best Practices:** Use compound assignment operators (`+=`, `*=`) to make code cleaner and less prone to variable name repetition typos.

> **Actionable Practice:** Write a snippet using compound operators to double a variable's value and then subtract 3.

Concept: Relational and Logical Operators

Simple Explanation: Relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) compare two values and yield a boolean result (`true` or `1`, `false` or `0`). Logical operators (`&&` AND, `||` OR, `!` NOT) combine multiple boolean conditions.

> ***Real-Life Analogy:*** *Relational operators are like security checkpoint comparisons ('Is height greater than 120cm?'). Logical operators are club admission rules: 'You can enter IF you have a ticket AND you are over 18' (`&&`).*

```
Syntax:
bool result = (a > b) && (c == 5);
bool notTrue = !result;
```

```
#include
using namespace std;

int main() {
    int age = 20;
    bool hasID = true;

    bool canEnterClub = (age >= 18) && hasID;
    bool isChildOrSenior = (age < 12) || (age > 65);

    cout << "Can Enter Club: " << canEnterClub << endl;
    cout << "Is Child or Senior: " << isChildOrSenior << endl;
    return 0;
}
```

Output:

```
Can Enter Club: 1
Is Child or Senior: 0
```

Line-by-Line Breakdown:

- **(age >= 18) && hasID**: Both conditions must evaluate to true for `canEnterClub` to be true.
- **(age < 12) || (age > 65)**: Evaluates to true if at least one condition is satisfied.

Real-World Application: Access control systems in secure corporate buildings verify badge validity (`hasBadge == true`) AND clearance level (`level >= 5`) using logical AND.

Common Mistakes: Using a single assignment equals `=` instead of the relational equality operator `==` inside conditional evaluations.

Best Practices: Use parentheses around individual relational expressions when combining them with logical operators to ensure crystal-clear evaluation order.

Actionable Practice: Write a boolean expression checking whether a number is strictly between 10 and 50.

Concept: Increment, Decrement, and Bitwise Operators

Simple Explanation: Increment (`++`) and decrement (`--`) operators increase or decrease a variable by 1, functioning in prefix or postfix modes. Bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`) manipulate data at the individual binary bit level.

> ***Real-Life Analogy:*** *Increment is clicking a mechanical counter forward by one click. Bitwise operations are like toggling individual electrical switches on a circuit board where each switch represents a binary 0 or 1.*

Syntax:

```
int x = 5;
int y = ++x; // Prefix: increments first, then assigns
int z = x++; // Postfix: assigns first, then increments
int bitAnd = 5 & 1; // Bitwise AND
```

```
#include
using namespace std;

int main() {
    int a = 5;
    int b = a++; // b is 5, then a becomes 6
    int c = ++a; // a becomes 7, then c is 7

    int bitwise = 5 & 3; // 5 (0101) & 3 (0011) = 1 (0001)

    cout << "b: " << b << ", c: " << c << ", Bitwise AND: " <<
bitwise << endl;
    return 0;
}
```

Output:

```
b: 5, c: 7, Bitwise AND: 1
```

Line-by-Line Breakdown:

- **int b = a++;**: Postfix increment. The original value of `a` (5) is assigned to `b`, and then `a` is incremented.
- **int c = ++a;**: Prefix increment. `a` is incremented first to 7, and then assigned to `c`.
- **5 & 3**: Compares binary bits `0101` and `0011`, resulting in `0001` (decimal 1).

Real-World Application: Bitwise operators are used extensively in embedded systems and graphics programming to pack multiple boolean flags into a single integer byte to save memory.

> **Common Mistakes:** Confusing prefix (`++x`) and postfix (`x++`) behaviors when embedded inside larger mathematical or assignment expressions.

> **Best Practices:** Avoid placing increment/decrement operators inside complex multi-variable expressions as it reduces code readability and can cause compiler-dependent evaluation quirks.

> **Actionable Practice:** Calculate the bitwise OR of 12 (`1100`) and 10 (`1010`) and write down the expected decimal outcome.

Concept: Ternary Operator and Operator Precedence

Simple Explanation: The ternary operator (`condition ? true_case : false_case`) acts as a shorthand inline `if-else` statement. Operator precedence determines the strict hierarchical order in which different operators are evaluated in complex expressions.

> ***Real-Life Analogy:*** *The ternary operator is like a quick traffic intersection sign: 'Is the light green? Go right (true), otherwise go left (false)'. Operator precedence is the mathematical PEMDAS/BODMAS rule applied to programming expressions.*

Syntax:
`string status = (score >= 50) ? "Pass" : "Fail";`

```
#include
using namespace std;

int main() {
    int num = 7;
    string result = (num % 2 == 0) ? "Even" : "Odd";

    // Precedence test: multiplication before addition
    int val = 10 + 5 * 2; // Evaluates as 10 + (5 * 2) = 20

    cout << "Number is: " << result << endl;
    cout << "Precedence result: " << val << endl;
    return 0;
}
```

Output:
```
Number is: Odd
Precedence result: 20
```

Line-by-Line Breakdown:

- **(num % 2 == 0) ? ...**: Evaluates whether `num` is even. If true, assigns "Even", else "Odd".
- **10 + 5 * 2**: Multiplication has higher precedence than addition, so `5 * 2` is calculated first.

Real-World Application: Ternary operators are widely used in UI rendering code to quickly toggle display states (e.g., `isLoggedIn ? "Logout" : "Login"`).

Common Mistakes: Nesting multiple ternary operators deeply, creating unreadable and hard-to-debug spaghetti code.

Best Practices: Always use explicit parentheses `()` in complex expressions to override or clarify operator precedence rather than relying entirely on default compiler hierarchy.

> **Actionable Practice:** Use a ternary operator to find the maximum of two numbers (`a` and `b`).

Concept: Variable Scope

Simple Explanation: Variable scope defines the region of a program where a variable is accessible. Local variables are declared inside a function or block and only exist locally. Global variables are declared outside all functions and are accessible globally throughout the file.

> ***Real-Life Analogy:*** *A local variable is like cash inside your personal wallet—only you can access it inside your room. A global variable is like a billboard in the town square—visible and accessible to everyone in the entire town.*

```
Syntax:
int globalVar = 100; // Global

void function() {
int localVar = 50; // Local
}
```

```
#include
using namespace std;

int globalScope = 500;

void display() {
    int localScope = 100;
    cout << "Global inside function: " << globalScope << endl;
    cout << "Local inside function: " << localScope << endl;
}

int main() {
```

```
    display();
    // cout << localScope; // Error: localScope is out of scope here
    cout << "Global in main: " << globalScope << endl;
    return 0;
}
```

Output:

```
Global inside function: 500
Local inside function: 100
Global in main: 500
```

Line-by-Line Breakdown:

- **int globalScope = 500;**: Declared globally, accessible by any function in this file.
- **int localScope = 100;**: Declared inside `display()`, destroyed immediately when `display()` finishes executing.

Real-World Application: Global variables are heavily restricted in modern software engineering because multiple parts of a program modifying the same global state simultaneously cause unpredictable bugs.

Common Mistakes: Trying to access a local variable outside of the curly brace block `{}` where it was declared.

Best Practices: Minimize the use of global variables completely. Pass variables as parameters between functions to maintain modular, clean, and bug-free code.

Actionable Practice: Declare a global variable and a local variable with the exact same name, and observe how shadowing works.

Visual Workflows & Architecture

Visualizing how data types occupy memory and how expressions are evaluated prevents subtle runtime bugs.

Diagram 1: Variable Storage in RAM

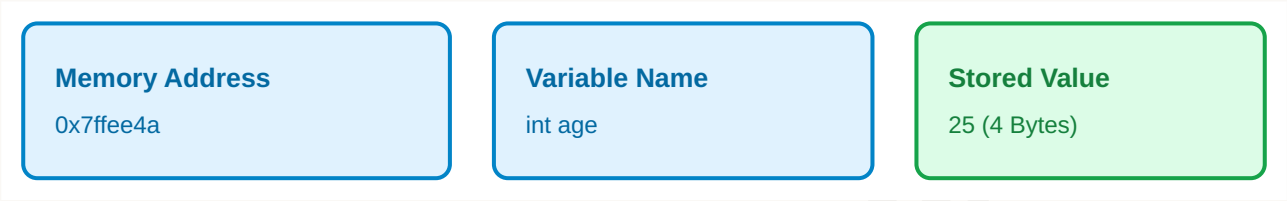


Diagram 2: Data Type Categories

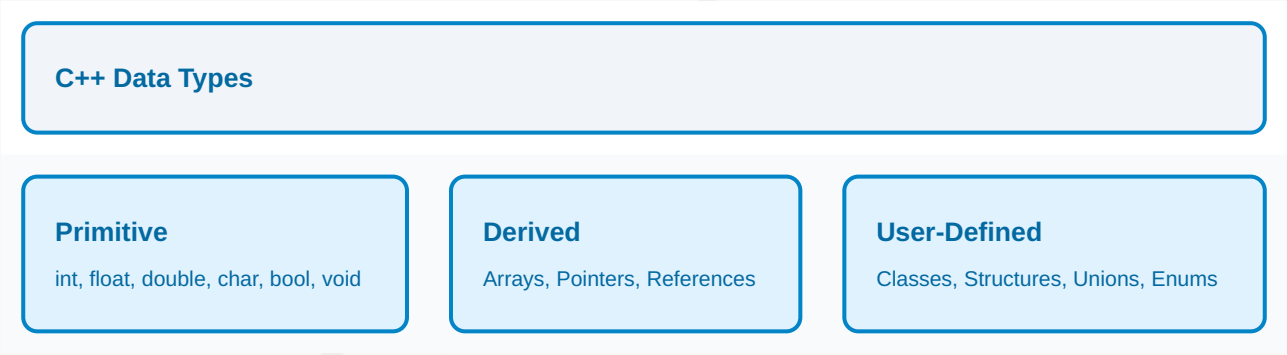


Diagram 3: Type Conversion Workflow

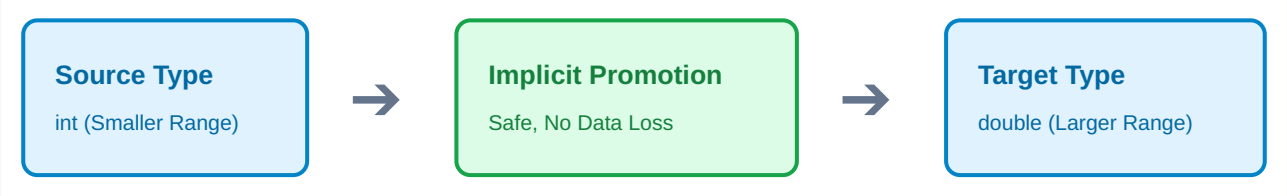


Diagram 4: Expression Evaluation Order (Precedence)



IT STUDY HUB

Comparative Analysis

Understanding subtle distinctions between data types and operators is essential for writing professional, bug-free C++ code.

Table 1: int vs float vs double

Feature	int	float	double
Data Type	Whole integers	Single-precision floating-point	Double-precision floating-point
Typical Size	4 Bytes (32 bits)	4 Bytes (32 bits)	8 Bytes (64 bits)
Precision	Exact (no decimals)	~6-7 decimal digits	~15-15 decimal digits
Use Case	Counters, loop indices, IDs	Graphics, basic measurements	Scientific computing, finance, high precision

Table 2: char vs string

Aspect	char	std::string
Storage	Single character enclosed in single quotes (`'A'`)	Sequence of characters enclosed in double quotes (`"Hello"`)
Memory Footprint	Exactly 1 Byte	Dynamic size (managed via STL class)
Header File	None required (built-in primitive)	Requires `#include <string>`

Table 3: signed vs unsigned

Modifier	signed	unsigned
Value Range	Negative, zero, and positive numbers	Strictly zero and positive numbers (0 to Max)
Bit Allocation	Uses the most significant bit for sign (`+` or `-`)	Uses all bits entirely for magnitude, doubling positive capacity

Table 4: Implicit vs Explicit Conversion

Aspect	Implicit Conversion	Explicit Conversion (Casting)
Initiation	Automatic by the compiler	Manual instruction by the programmer
Safety	Generally safe (widening conversions)	Can be risky (narrowing conversions leading to data loss)
Syntax	`double d = 10;`	`int i = static_cast<int>(3.14);`

Table 5: `=` vs `==` and `&&` vs `||`

Operator Pair	Primary Operator	Secondary Operator
Equality Check	`=` (Assignment: stores value)	`==` (Relational: compares equality)
Logical Evaluation	`&&` (Logical AND: both true)	`||` (Logical OR: at least one true)

HANDS-ON PROJECT: BASIC C++ CALCULATOR

In this hands-on project, we combine variables, primitive data types, input/output streams, and arithmetic operators to build a robust, interactive command-line calculator.

Project Specification

Problem Statement: Write a complete C++ program that prompts the user to enter two numbers and an arithmetic operator (`+`, `-`, `*`, `/`). The program must perform the requested calculation, handle division-by-zero safely, and display clear, formatted results utilizing appropriate data types.

```
#include <iostream>
using namespace std;

int main() {
    double num1, num2, result = 0;
    char op;
    bool validOperation = true;

    cout << "========================================" << endl;
    cout << "      IT STUDY HUB C++ CALCULATOR       " << endl;
    cout << "========================================" << endl;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter operator (+, -, *, /): ";
    cin >> op;

    cout << "Enter second number: ";
    cin >> num2;

    switch(op) {
        case '+':
            result = num1 + num2;
```

```
            break;
        case '-':
            result = num1 - num2;
            break;
        case '*':
            result = num1 * num2;
            break;
        case '/':
            if(num2 != 0) {
                result = num1 / num2;
            } else {
                cout << "Error: Mathematical Division by Zero!" <<
endl;
                validOperation = false;
            }
            break;
        default:
            cout << "Error: Invalid Operator Selected!" << endl;
            validOperation = false;
            break;
    }

    if(validOperation) {
        cout << "----------------------------------------" << endl;
        cout << "Result: " << num1 << " " << op << " " << num2 << "
= " << result << endl;
        cout << "----------------------------------------" << endl;
    }

    return 0;
}
```

Expected Output:

```
========================================
      IT STUDY HUB C++ CALCULATOR
========================================
Enter first number: 50
Enter operator (+, -, *, /): /
Enter second number: 4
----------------------------------------
```

```
Result: 50 / 4 = 12.5
----------------------------------------
```

Step-by-Step Explanation:

- We declare `double num1, num2, result;` to support both whole and decimal numbers accurately.
- We use `char op;` to capture the user's chosen mathematical operator symbol.
- We use a `switch` statement (previewing control flow) to evaluate which arithmetic operation to execute.
- We incorporate a safety check (`if(num2 != 0)`) to prevent catastrophic runtime division-by-zero errors.
- Finally, we print a clean formatted receipt showing the full equation and computed result.

Practice Challenges:

- Extend the calculator to support the modulus operator (`%`) by casting inputs to integers.
- Add an exponentiation calculation feature using the `` header (`pow(base, exp)`).

Module Assessments

Verify your mastery of variables, data types, and operators with these rigorous assessment questions.

15+ Practice Questions

1. What is the difference between a variable and a constant in C++?
2. Why must variables be declared with a data type before usage?
3. What is the output of `sizeof(double)` on a standard 64-bit system?
4. Differentiate between `signed` and `unsigned` integer modifiers.
5. What is implicit type promotion? Give an example.
6. Why is explicit type casting (`static_cast`) necessary when dividing two integers to get a decimal result?
7. Evaluate the expression: `int x = 10 + 2 * 5 - 4 / 2;` and explain the precedence order.
8. What is the difference between prefix (`++x`) and postfix (`x++`) increment operators?
9. Explain what happens when you assign a floating-point number to an integer variable.
10. Write a ternary expression that checks if a variable `score` is greater than or equal to 40 ("Pass" or "Fail").
11. What is variable scope? Explain local vs global scope.
12. What is the output of `cout << (5 > 3 && 10 < 20);`?
13. Explain the purpose of the bitwise AND (`&`) operator.
14. Why should global variables be avoided in large software projects?
15. Write a snippet showing how to declare a constant float for Pi.

20+ Viva Questions (With Answers)

Q1: What is a variable?

> A named memory location used to store data that can be modified during program execution.

Q2: How do you declare a constant in C++?

> Using the `const` keyword (e.g., `const int MAX = 100;`).

Q3: What is the default size of an `int` on 64-bit systems?

> 4 bytes (32 bits).

Q4: What is the difference between `float` and `double`?

> `float` provides single precision (4 bytes), whereas `double` provides double precision (8 bytes) with more decimal places.

Q5: What does the `sizeof` operator return?

> The size in bytes of a given data type or variable.

Q6: What is type casting?

> Converting a variable from one data type to another.

Q7: What is implicit conversion?

> Automatic conversion performed safely by the compiler without programmer intervention.

Q8: What is explicit conversion?

> Manual conversion forced by the programmer using casting operators like `static_cast`.

Q9: What does the modulus operator (`%`) do?

> Returns the remainder of integer division.

Q10: Can you use modulus on `float` or `double`?

> No, modulus only operates on integer types.

Q11: What is the difference between `=` and `==`?

> `=` is the assignment operator; `==` is the relational equality comparison operator.

Q12: What does `&&` represent?

> Logical AND operator (evaluates to true only if all conditions are true).

Q13: What is short-circuit evaluation in logical operators?

> In an `&&` expression, if the first condition is false, the second is not evaluated because the overall result is already known to be false.

Q14: What is the ternary operator syntax?

> `condition ? expression_if_true : expression_if_false;`

Q15: What is the difference between local and global scope?

> Local variables are restricted to the block/function where declared; global variables are accessible throughout the entire file.

Q16: What is an unsigned integer?

> An integer modifier that restricts values to zero and positive numbers, doubling the upper positive limit.

Q17: What is the purpose of `void` data type?

> Represents the absence of value or type (commonly used for functions that do not return a value).

Q18: What is operator precedence?

> The hierarchical rule determining the order in which different operators are evaluated in an expression.

Q19: What is a bitwise shift operator?

> `<<` (left shift) and `>>` (right shift) operators that shift the binary bits of a number by a specified number of positions.

Q20: What is a narrowing conversion warning?

> A compiler warning triggered when assigning a larger data type (like `double`) to a smaller data type (like `int`), indicating potential data loss.

10+ Technical Interview Questions

1. **Why does `5 / 2` evaluate to `2` instead of `2.5` in C++? How do you fix it?** Explain integer division truncation and how casting one operand to `double` forces floating-point division.

2. **What is integer overflow, and what happens when an `unsigned int` exceeds its maximum limit?** Discuss wrap-around behavior where the value rolls over to `0`.
3. **When should you use `static_cast` instead of C-style casting?** Explain type safety, compiler checkability, and readability benefits of modern C++ casting.
4. **Explain short-circuit evaluation in logical AND (`&&`) and OR (`||`) operators.** Provide an example where short-circuiting prevents null-pointer dereferencing or division-by-zero crashes.
5. **What is the difference between `const int* p`, `int* const p`, and `const int* const p`? (Preview of pointers)** Discuss what parts of the pointer and data are immutable.
6. **How does the compiler allocate memory for local vs global variables?** Explain stack allocation for local block variables and data-segment allocation for globals.
7. **What are bitwise flags, and how are they used in systems programming?** Explain how combining powers of 2 with bitwise OR (`|`) allows packing multiple flags into a single integer.
8. **Why is `double` preferred over `float` in modern CPU architectures despite taking more memory?** Discuss modern 64-bit register optimization where double-precision floats execute just as fast or faster than singles.
9. **What is the hazard of uninitialized variables in C++?** Explain how reading uninitialized memory accesses leftover garbage values, leading to nondeterministic bugs.
10. **How do operator precedence and associativity work together when two operators of the same precedence appear in an expression?** Explain left-to-right vs right-to-left associativity rules.

Module 2 Wrap-Up

Key Terms Glossary

- **Variable:** A named memory location holding mutable data.
- **Constant:** Immutable data declared with `const`.
- **Primitive Type:** Fundamental built-in data types (`int`, `double`, `char`, etc.).
- **Modifier:** Keywords (`signed`, `unsigned`, `short`, `long`) altering base type ranges.
- **Type Casting:** Converting data from one type to another (implicit or explicit).
- **Operator Precedence:** The strict rule determining expression evaluation order.
- **Scope:** The region of code where a variable remains accessible.

Quick Revision Summary

Variables store mutable data in RAM, while constants (`const`) are immutable. Primitive types include integers (`int`), decimals (`float`, `double`), characters (`char`), and booleans (`bool`). Modifiers like `unsigned` double positive capacity. Type casting handles conversions either implicitly (promotions) or explicitly (`static_cast`). Operators span arithmetic, assignment, relational, logical, bitwise, and ternary evaluations. Operator precedence dictates evaluation order, while scope restricts variable accessibility to local blocks or global files.

Beginner Coding Challenges

Solidify your understanding of Module 2 by attempting these practice challenges:

- **Challenge 1: Temperature Converter.** Write a program that takes a temperature in Celsius as a `double`, converts it to Fahrenheit using the formula ***F = (C * 9.0/5.0) + 32***, and prints both with clean formatting.
- **Challenge 2: The Bitwise Mask.** Declare an integer, apply bitwise shifts and masking operations to extract and display its individual byte components.
- **Challenge 3: Grade Evaluator.** Take a student's numerical score (0-100) and use a nested ternary operator to assign and print a letter grade (`A`, `B`, `C`, `D`, `F`).

CONGRATULATIONS ON COMPLETING MODULE 2!

WHAT COMES NEXT:

MODULE 3 — CONTROL FLOW & FUNCTIONS